FusionDrive

# ADAS Sensor Fusion on QNX RTOS

Deterministic collision-risk decisions from an IMU, an ultrasonic sensor and a USB camera on a Raspberry Pi 4 running QNX SDP 8.0

| | |
|---|---|
| Team | FusionDrive |
| Team ID | QNXHACK-HYD-TEAM-026 |
| Team members | Sakilam Adithya · Dubbaka Manoj Goud · Soma Praneeth Kumar |
| Event | QNX India Hackathon (QNX Everywhere), State / Cluster level |
| Platform | Raspberry Pi 4 Model B · QNX SDP 8.0 Quick Start Target Image · Momentics IDE 8.0 |
| Sensors | MPU6050 IMU (I2C) · HC-SR04 ultrasonic (GPIO + interrupt) · Logitech C920 (USB, QNX Sensor Framework) |
| Document | Project report · October 2026 |

**Contents**


1. Introduction 2
2. Hardware 2
3. System architecture 4
4. Software design 6
5. QNX mechanisms 8
6. Fault tolerance 9
7. Setup and build 9
8. Running and demonstration 10
9. Results 10
10. Limitations and future work 14
11. Development notes 15

Appendix A. Repository layout 15

Appendix B. Troubleshooting 15

# 1. Introduction

An Advanced Driver Assistance System (ADAS) has to turn several imperfect sensors into one reliable picture of the road ahead, and it has to do so on time, every time. A collision warning that arrives late, or that is computed from a stale reading which looks fresh, is as dangerous as no warning at all.

FusionDrive is a working prototype of such a controller built on the QNX Neutrino RTOS. Three real sensors run as independent QNX processes and send timestamped measurements to a fusion server through kernel message passing. Every 20 ms the server fuses them with a Kalman filter, computes the time-to-collision (TTC) and decides **SAFE, WARNING, CRITICAL or EMERGENCY**. When a sensor fails, the microkernel reports it and the system keeps deciding with the sensors that remain.

## 1.1 Problem statement

- A vehicle ahead brakes suddenly; the controller must estimate the gap and closing speed and react in time.
- At 54 km/h a car closes 30 cm every 20 ms, so the decision period must be short and guaranteed, not "usually fast".
- Sensors run at different rates (IMU 200 Hz, ultrasonic 15 Hz, camera 30 fps) and fail in different ways.
- A single faulty or crashed sensor must not stop or corrupt the safety function.

## 1.2 Objectives and success metrics

| Objective | Metric | Achieved |
|---|---|---|
| Deterministic decision loop | 20 ms period, zero deadline misses | 0 misses over 14 300+ cycles |
| Low-latency IPC | sensor to fusion latency | 7-20 us per message |
| Honest timing | every sample timestamped on one clock | CLOCK_MONOTONIC in all three nodes |
| Fault isolation | failed sensor detected within one cycle | kernel DISCONNECT pulse, OFFLINE at once |
| Graceful degradation | keep deciding with remaining sensors | camera carries the track when ultrasonic dies |
| Efficiency | low CPU load | 79 % idle on 4 cores (5 s trace) |

# 2. Hardware

## 2.1 Bill of materials

| Component | Purpose | Interface |
|---|---|---|
| Raspberry Pi 4 Model B | target running QNX 8.0 | - |
| MPU6050 (GY-521) | ego-motion: acceleration, rotation, temperature | I2C1, 400 kHz, address 0x68 |
| HC-SR04 | distance to the object ahead | GPIO23 trigger, GPIO24 echo, GPIO IRQ 145 |
| Logitech C920 HD Pro | object detection, image-based TTC | USB 3.0, QNX Sensor Framework unit 4 |
| Resistors 1 kOhm + 2 kOhm | echo voltage divider 5 V to 3.3 V | - |
| Development PC | QNX SDP 8.0, Momentics IDE | Ethernet (link-local 169.254.x.x) |

## 2.2 Wiring

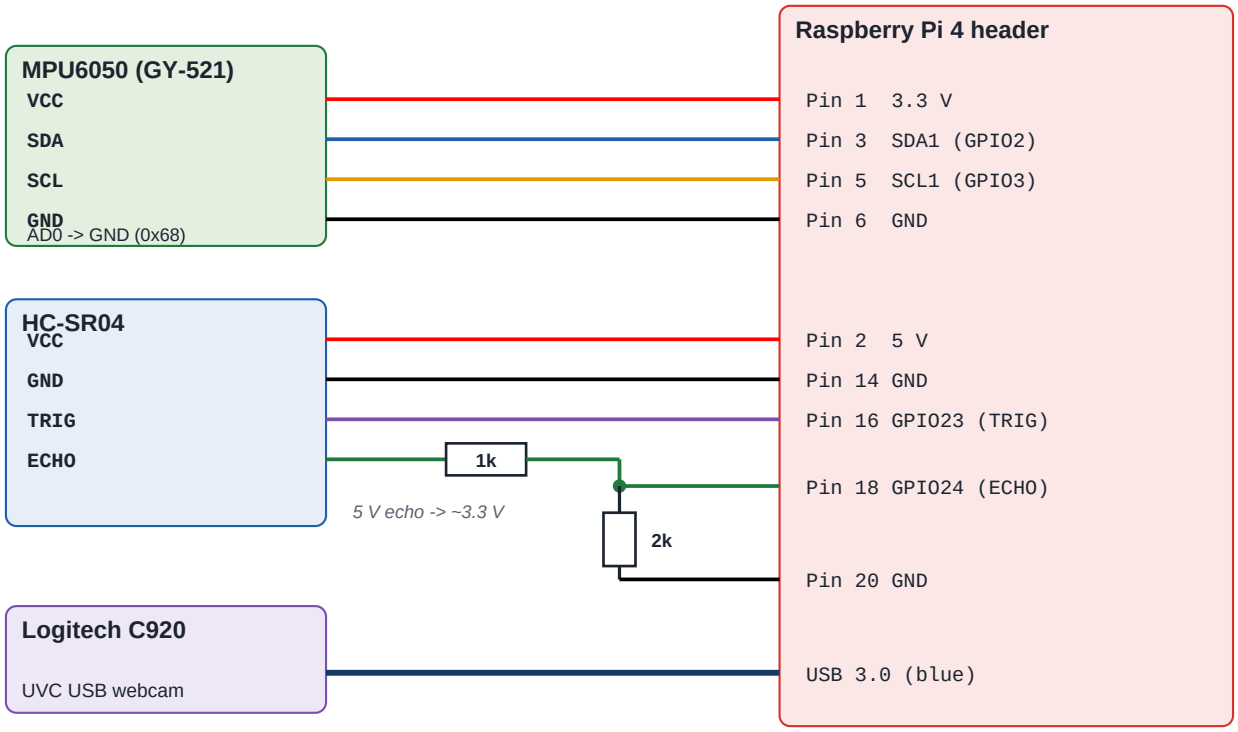



**Figure 1.** Wiring of the three sensors to the Raspberry Pi 4. The HC-SR04 echo output is 5 V and must pass through the divider.

**Safety:** wire with the Pi powered off. Never connect the HC-SR04 ECHO pin directly to a GPIO. The C920 must be in a blue USB 3.0 port.

# 3. System architecture

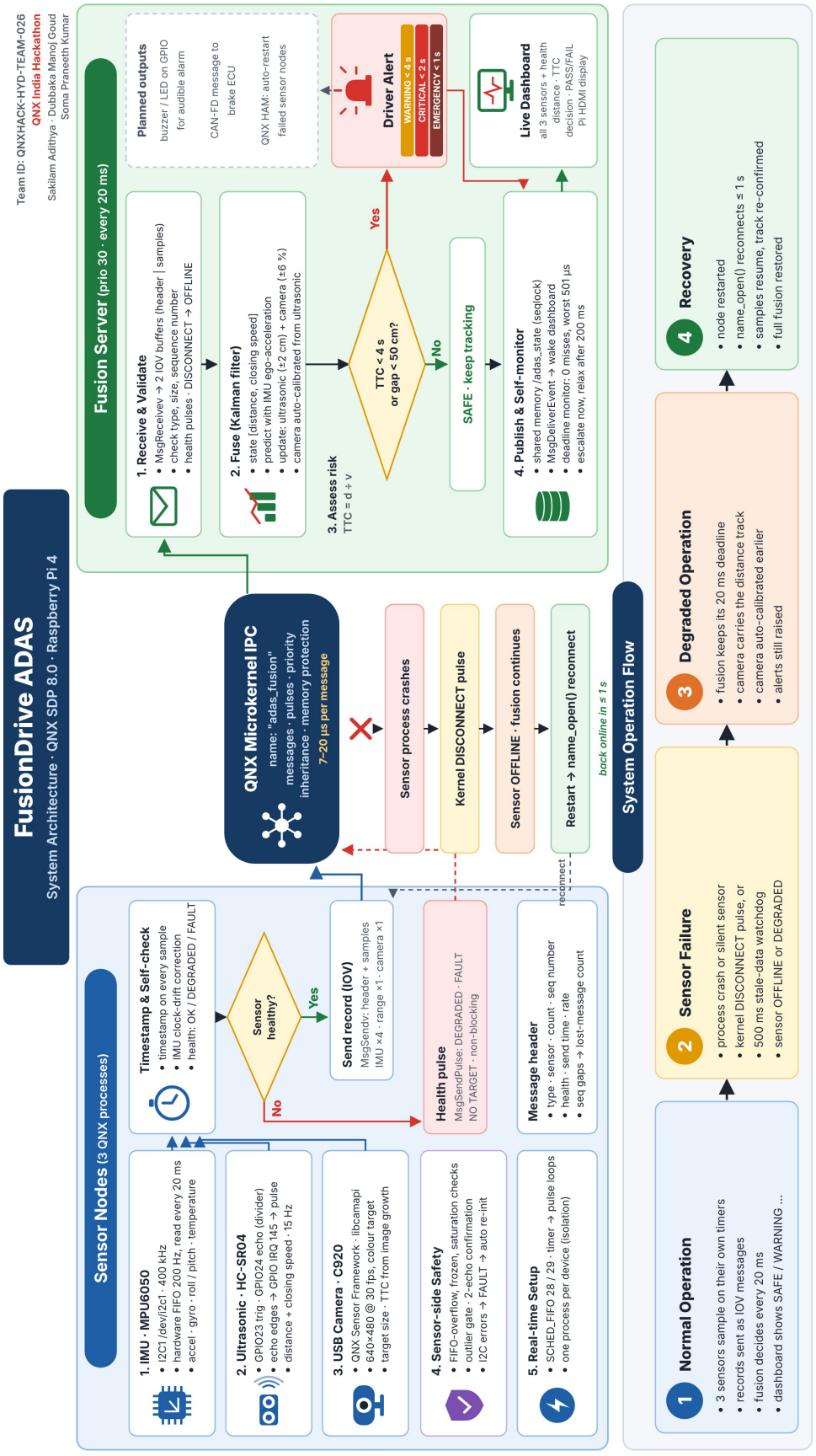



**Figure 2.** System overview: sensor nodes, QNX microkernel IPC, fusion server, failure chain and operation flow (rotate page to read).

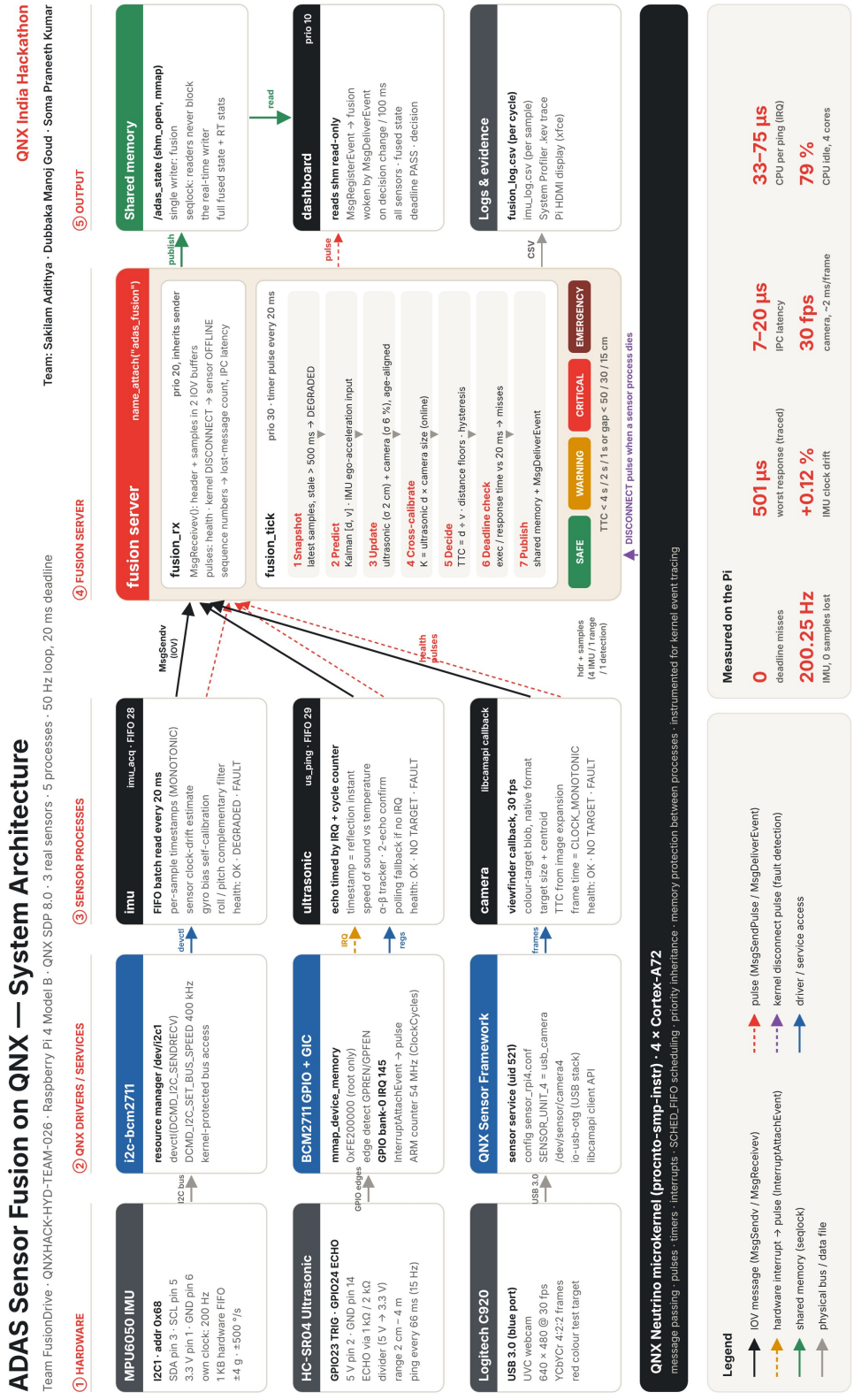



**Figure 3.** Detailed architecture: hardware, QNX drivers and services, sensor processes, fusion server pipeline, outputs and the microkernel.

### 3.1 Process structure

The system is split along device boundaries: one process owns each device, and a separate fusion server owns the decision. The QNX microkernel isolates the processes in memory, so a crash in one sensor node cannot corrupt another or the fusion server.

| Process | Device access | Main thread / priority | Role |
|---|---|---|---|
| imu | I2C resource manager /dev/i2c1 (devctl) | imu_acq, SCHED_FIFO 28 | FIFO batch acquisition, timestamps, drift, health |
| ultrasonic | GPIO registers (mmap_device_memory), IRQ 145 | us_ping, SCHED_FIFO 29 | echo timing, tracker, outlier rejection |
| camera | libcamapi via QNX Sensor Framework | viewfinder callback | colour target, size, expansion TTC |
| fusion | - | fusion_tick 30, fusion_rx 20 (inherits) | Kalman fusion, TTC, decision, deadline monitor |
| dashboard | shared memory (read-only) | main, 10 | live display on the Pi screen |
| sensor (QNX) | USB camera stack io-usb-otg | QNX service, uid 521 | publishes /dev/sensor/camera4 |

# 4. Software design

### 4.1 IMU node (MPU6050)

- The MPU6050 samples itself at exactly 200 Hz into its 1 KB hardware FIFO; a QNX timer pulse wakes the priority-28 thread every 20 ms to drain it in one burst (about 4 frames). No sample is lost even if the reader is late.
- Every sample gets a reconstructed CLOCK_MONOTONIC timestamp from its FIFO position and the measured sample period.
- The real sensor rate is estimated against the QNX clock (measured 200.249 Hz, drift +0.12 %).
- Start-up calibration of gyro bias and gravity scale; rejected and retried if the board moves.
- Health monitor: WHO_AM_I, FIFO overflow, frozen sensor, saturation, gravity plausibility, I2C errors with automatic re-initialisation.

### 4.2 Ultrasonic node (HC-SR04)

- A 12 us trigger pulse starts a ping every 66 ms (15 Hz, above the 60 ms the HC-SR04 needs).
- The BCM2711 edge detector raises GPIO bank-0 interrupt 145 on both echo edges; InterruptAttachEvent turns it into a pulse, so the thread sleeps instead of busy-polling. CPU per ping fell from about 7 000 us (polling) to 33-75 us.
- Edges are timed with the 54 MHz ARM cycle counter; the range is timestamped at the reflection instant.
- Speed of sound is temperature-compensated; an alpha-beta tracker gives distance and closing speed.
- A track starts only when two echoes agree; spikes are rejected by an innovation gate; lost-echo tails are skipped, not waited for.

## 4.3 Camera node (Logitech C920)

- Frames arrive through the QNX camera library (libcamapi) from the Sensor Framework service at 640 x 480, 30 fps.
- A colour test target (red by default) is detected directly in the camera's native YCbYCr format, sampling every second pixel and row (about 2 ms per frame).
- TTC is computed from image expansion: with s = sqrt(target area), TTC = 1 / (d ln s / dt). It needs no distance and no calibration.
- TTC is only published after 8 consecutive tracked frames, which removed a false spike seen when a target first appears.

## 4.4 Fusion server

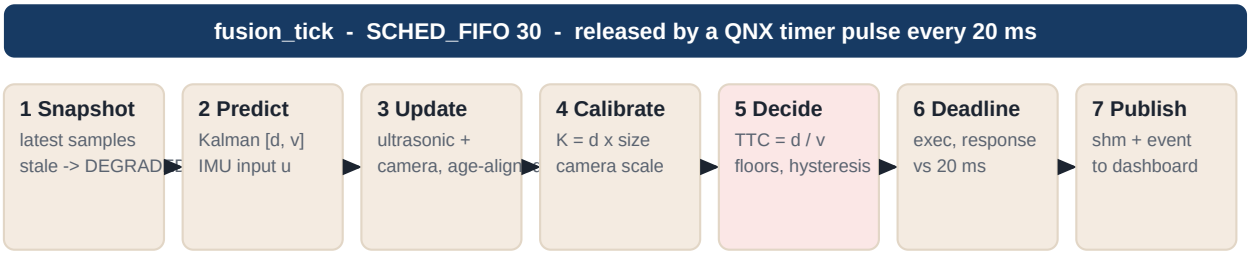



**Figure 4.** One fusion cycle. The whole pipeline runs inside the 20 ms period released by a timer pulse.

- **State:** x = [distance d, closing speed v]; prediction uses the IMU forward acceleration (gravity removed) as control input.
- **Updates:** ultrasonic range (sigma about 2 cm) and camera distance (sigma about 6 %), each shifted to "now" by its measurement age; 4-sigma innovation gate.
- **Online cross-calibration:** while both sensors see the target, K = d_ultrasonic x size_camera is learned, so the camera alone can provide distance later.
- **Fallback:** with no distance track at all, the camera's own expansion TTC still drives the decision.

## 4.5 Decision logic




**Figure 5.** Decision ladder. The more severe of the time-based and distance-based level is used.

## 4.6 Dashboard

The dashboard maps the shared-memory segment read-only and copies a consistent snapshot with a seqlock, so it never blocks the real-time writer. It registers a pulse event with fusion (MsgRegisterEvent) and sleeps until fusion delivers it on a decision change or every 100 ms. Each frame is written in one buffered write, so the screen never shows a half-drawn frame.

# 5. QNX mechanisms

| Mechanism | API | Where it is used |
|---|---|---|
| Named server | name_attach / name_open | fusion registers "adas_fusion"; clients start in any order |
| IOV message passing | MsgSendv / MsgReceivev | header + sample array as one two-part message, no repacking |
| Pulses | MsgSendPulse, timer sigevents | timer ticks, health changes, interrupt events |
| Kernel disconnect pulse | _PULSE_CODE_DISCONNECT | fusion learns instantly that a sensor process died |
| Event delivery | MsgRegisterEvent, MsgDeliverEvent | fusion wakes the dashboard only when needed |
| Shared memory | shm_open, mmap, seqlock | /adas_state: lock-free publication of the fused state |
| Interrupts | InterruptAttachEvent, InterruptUnmask | GPIO IRQ 145 for ultrasonic echo edges |
| Resource manager | devctl on /dev/i2c1 | I2C access through the QNX driver |
| Scheduling | SCHED_FIFO, pthread attributes | priorities 30 / 29 / 28 / 20 / 10 |
| Priority inheritance | message passing, mutexes | fusion_rx runs at the sender's priority |
| Timers | timer_create + pulse | every periodic loop; no sleep() in real-time paths |

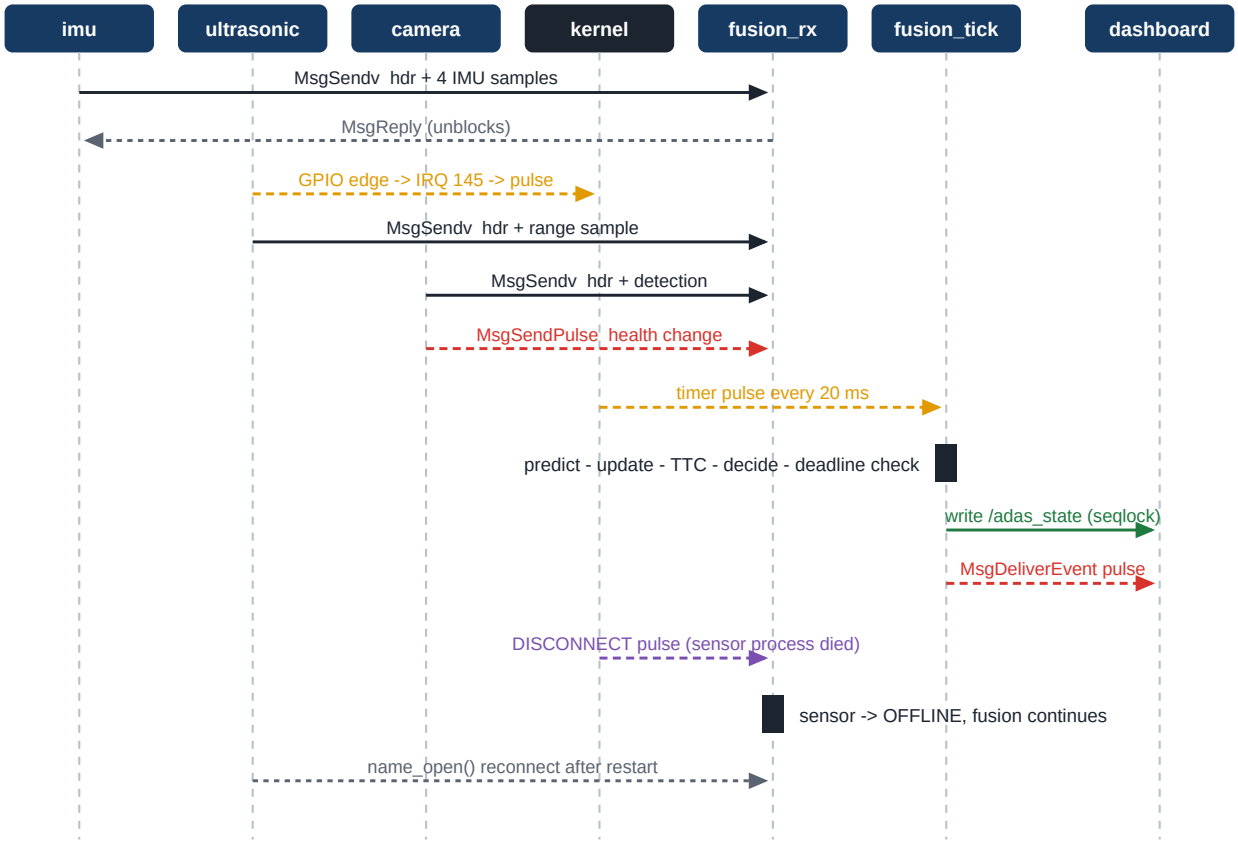
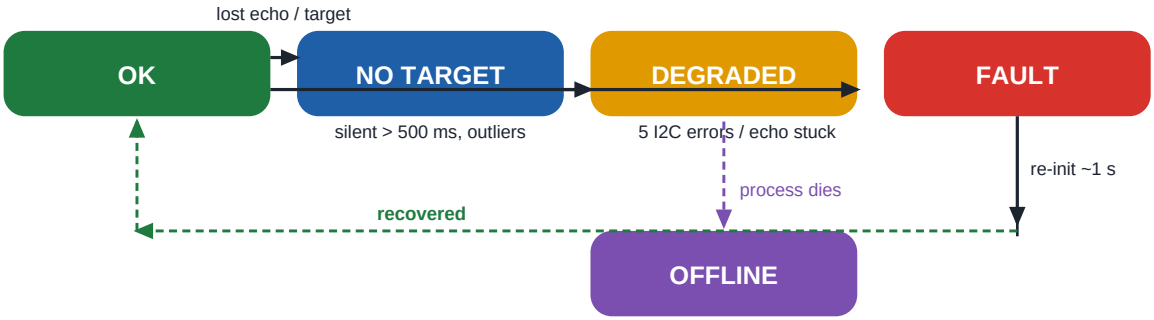



**Figure 6.** Message and pulse sequence between the processes during normal operation and a sensor failure.

# 6. Fault tolerance




*OFFLINE comes from the kernel DISCONNECT pulse; restart -> name_open() -> OK*

**Figure 7.** Sensor health states as seen by the fusion server.

| Fault | Detection | Response | Observed |
|---|---|---|---|
| Sensor process killed | kernel DISCONNECT pulse | OFFLINE in the same cycle; fusion continues | disconnects 1, misses 0 |
| Process restarted | name_open() every 1 s | reconnects, data resumes | online in <= 1 s |
| Sensor silent | 500 ms stale-data watchdog | DEGRADED, excluded from the filter | by design |
| Lost / stray echoes | 2-echo confirmation, gate | stray echo ignored | stray 0.33 m echo rejected |
| IMU I2C errors | 5 consecutive errors | FAULT, automatic re-init | 0 errors in 4 025 samples |
| Decision chatter | hysteresis | escalate at once, relax after 200 ms | no flicker |
| CPU overload | deadline monitor | priority scheduling keeps fusion on time | load test, 0 misses |

# 7. Setup and build

## 7.1 Target and host

- Write the QNX 8.0 Quick Start Target Image for Raspberry Pi 4 to the SD card (Raspberry Pi Imager), boot, log in as qnxuser.
- Install QNX SDP 8.0 and Momentics IDE; add the target (Target Navigator, IP of the Pi).
- For the camera install the Software Center package **QNX SDP 8.0 Sensor Framework Camera Library** (com.qnx.qnx800.target.sf.camapi).
- Check the I2C buses with `ls /dev/i2c*` (expect /dev/i2c1).

## 7.2 Building

Each program is one Momentics project (QNX Executable, aarch64le) with a single source file in src/. The shared header common/adas_msg.h is added through the include path.

```
# in each project Makefile
INCLUDES += -I../common
LIBS     += -lm            # camera: LIBS += -lcamapi -lm

# or from a shell with the SDP environment loaded
for p in fusion imu ultrasonic camera dashboard; do make -C $p; done
```

### 7.3 Camera service

The QNX image starts the camera service from /etc/startup/post_startup.sh only if a display was present at boot. Start it by hand when /dev/sensor/camera4 is missing (run_all.sh does this automatically):

```
C=/etc/config/sensor/sensor_rpi4.conf
sudo sensor -U 521:521 -b external -r /data/share/sensor -c $C
ls /dev/sensor/            # camera1 camera2 camera4 data1  (camera4 = USB camera)
```

## 8. Running and demonstration

```
cd ~
sudo sh run_all.sh                                    # starts sensor service, fusion, imu,
                                                      # ultrasonic, camera, dashboard

# second terminal tab: fault injection
sudo slay ultrasonic                                  # ULTRASONIC -> OFFLINE
sudo sh -c './ultrasonic > ultrasonic.out 2>&1 &'     # back online in ~1 s

# CPU-load test: 4 busy threads below fusion priority
sudo slay fusion
sudo sh -c './fusion -l 4 -p 5 -o fusion_log.csv > fusion.out 2>&1 &'

# stop everything
sudo slay dashboard fusion imu ultrasonic camera
```

| Program | Options |
|---|---|
| fusion | -o log.csv -l hogs -p hog_prio -A forward_axis -t seconds |
| imu | -d /dev/i2c1 -a 0x68 -t seconds -o imu.csv -c (skip calibration) |
| ultrasonic | -g trig_gpio -e echo_gpio -T temp_C -r rate_hz -I irq\|0 (0 = polling) |
| camera | -u unit -c red\|green\|blue -w (video window) -a min_area_% |
| dashboard | -p (plain output) |

### 8.1 Demonstration sequence

- Start the system and show all three sensors OK on the dashboard.
- Approach demo: move a red object from about 1.5 m towards the sensors: SAFE, WARNING, CRITICAL, EMERGENCY.
- Kill the ultrasonic: OFFLINE and disconnects + 1 while fusion keeps running; the calibrated camera keeps the track.
- Restart it: reconnection within a second.
- CPU-load test: deadline monitor still PASS with 0 misses.

## 9. Results

| Metric | Result |
|---|---|
| Fusion deadline misses | 0 over 14 300+ cycles |
| Fusion execution time per cycle | 2-6 us |
| Fusion worst response time | 501 us in the traced run (<= 9.7 ms in any run), deadline 20 ms |
| IPC latency sensor -> fusion | 7-20 us |
| IMU | 200.249 Hz, drift +0.12 %, 0 samples lost, worst FIFO read 1.98 ms |
| Ultrasonic | 15.00 Hz, +/-0.5 cm at 1.2 m, CPU 33-75 us per ping (interrupt) |
| Camera | 30 fps, about 2 ms processing, 2 ms capture-to-result |

| Metric | Result |
|---|---|
| System Profiler (5 s) | 79.2 % idle on 4 cores, 0 of 1688 buffers dropped, camera 5.3 %, ultrasonic 2.4 % of one core |

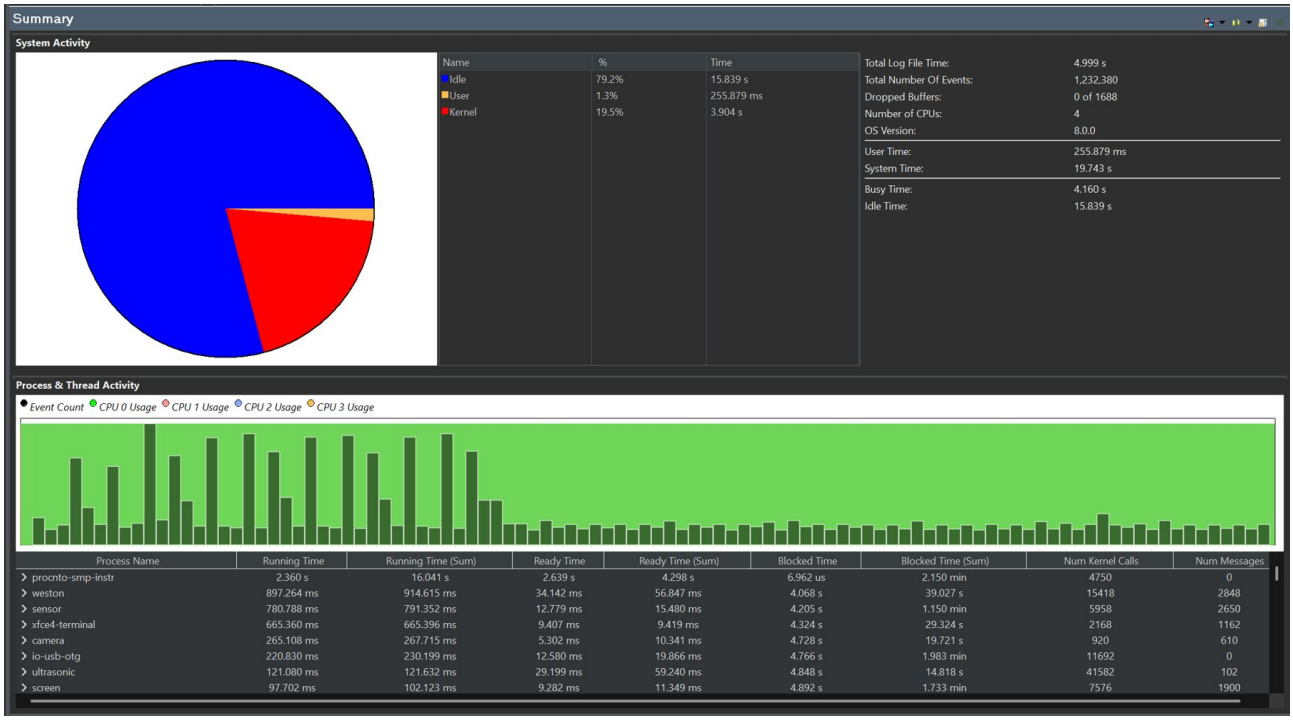



**Figure 8.** QNX Momentics System Profiler: 5 s kernel event trace with all processes running.

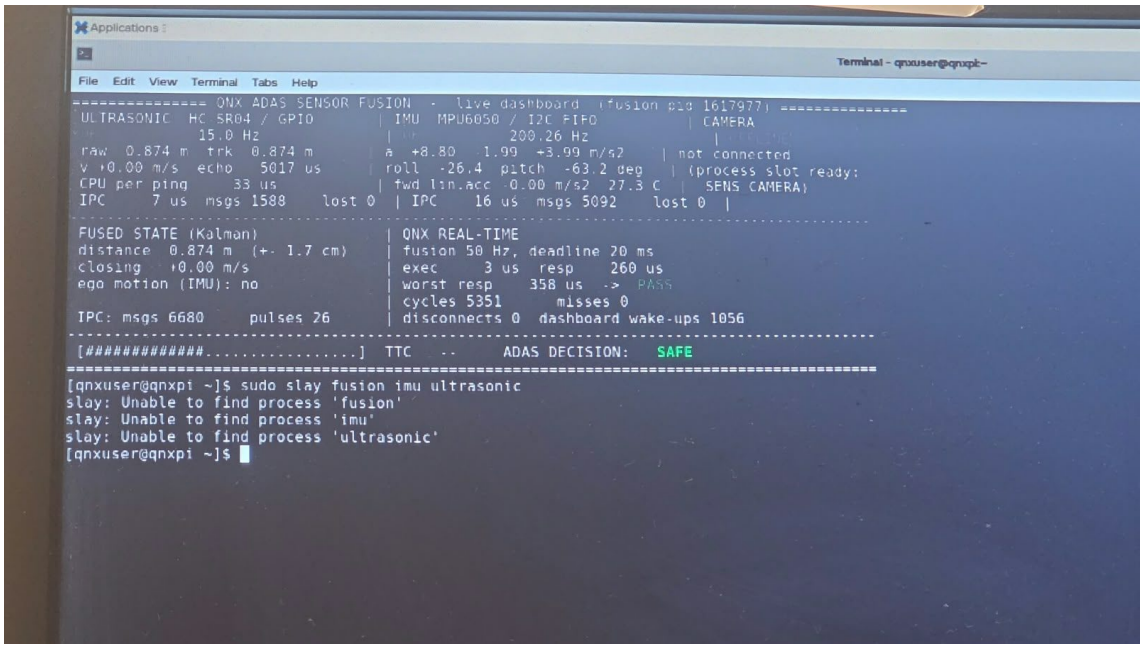



**Figure 9.** Live dashboard on the Pi: all sensors, fused state, real-time statistics and the decision.

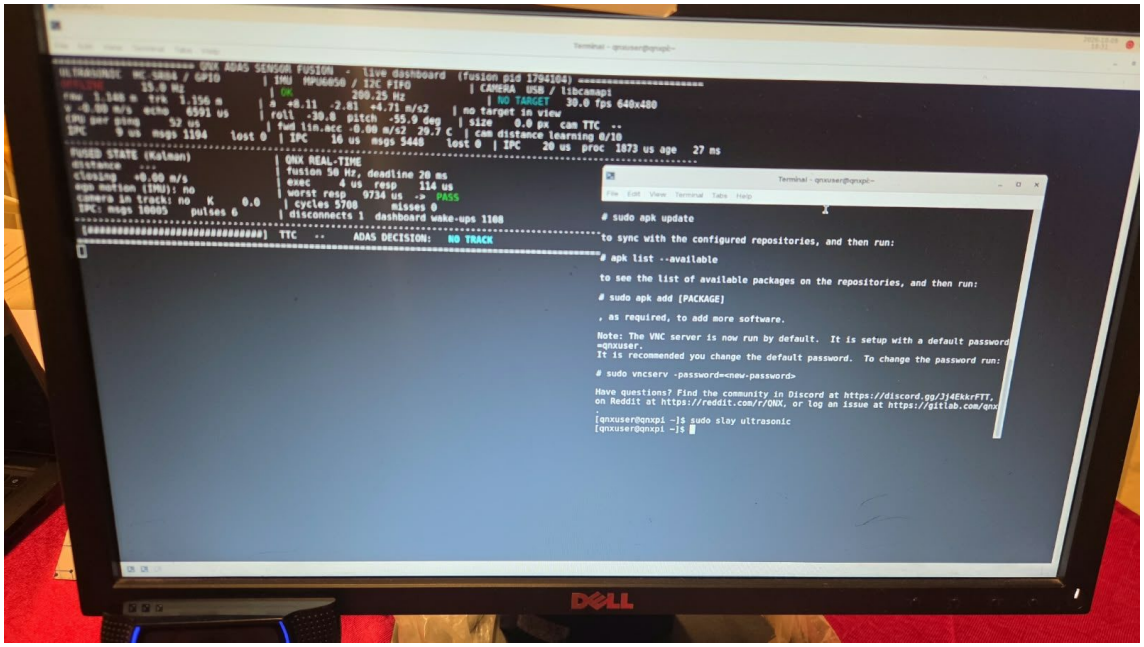



**Figure 10.** Fault demonstration: the ultrasonic process was killed; fusion keeps running and counts one disconnect.

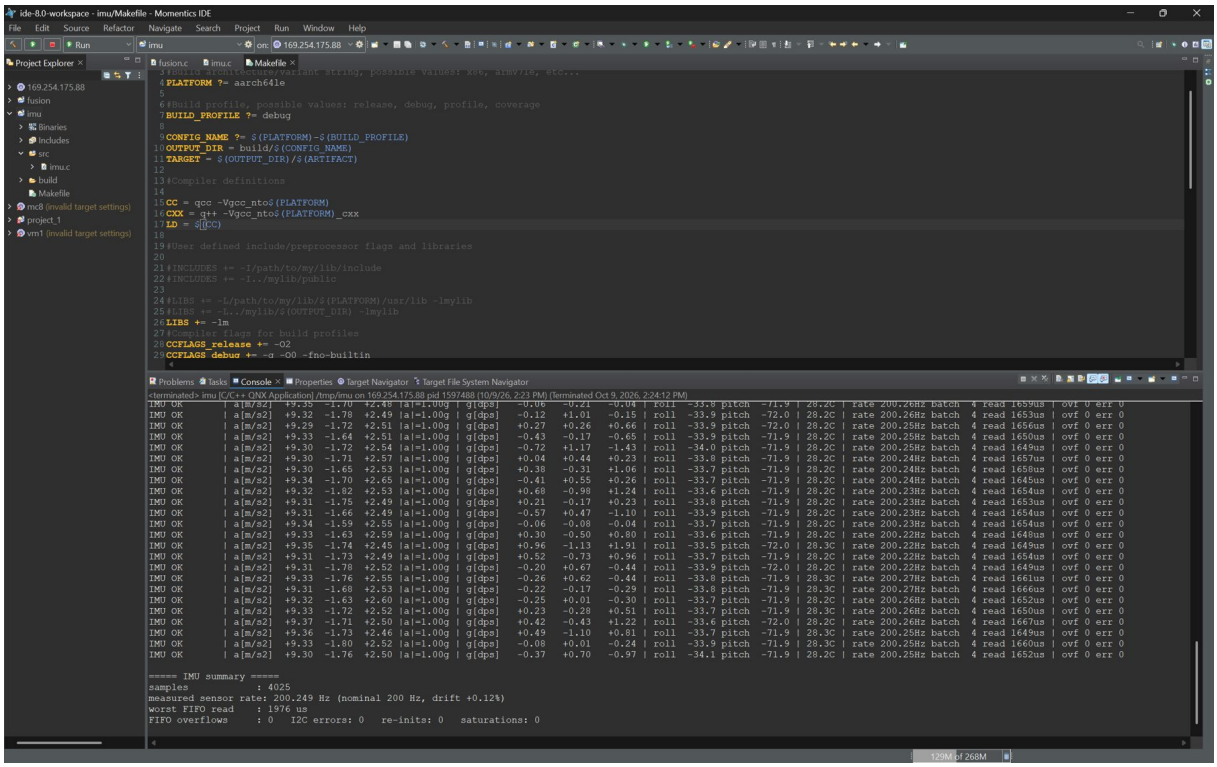



**Figure 11.** IMU node, 20 s run: 4 025 samples, measured rate 200.249 Hz (drift +0.12 %), no overflows or I2C errors.

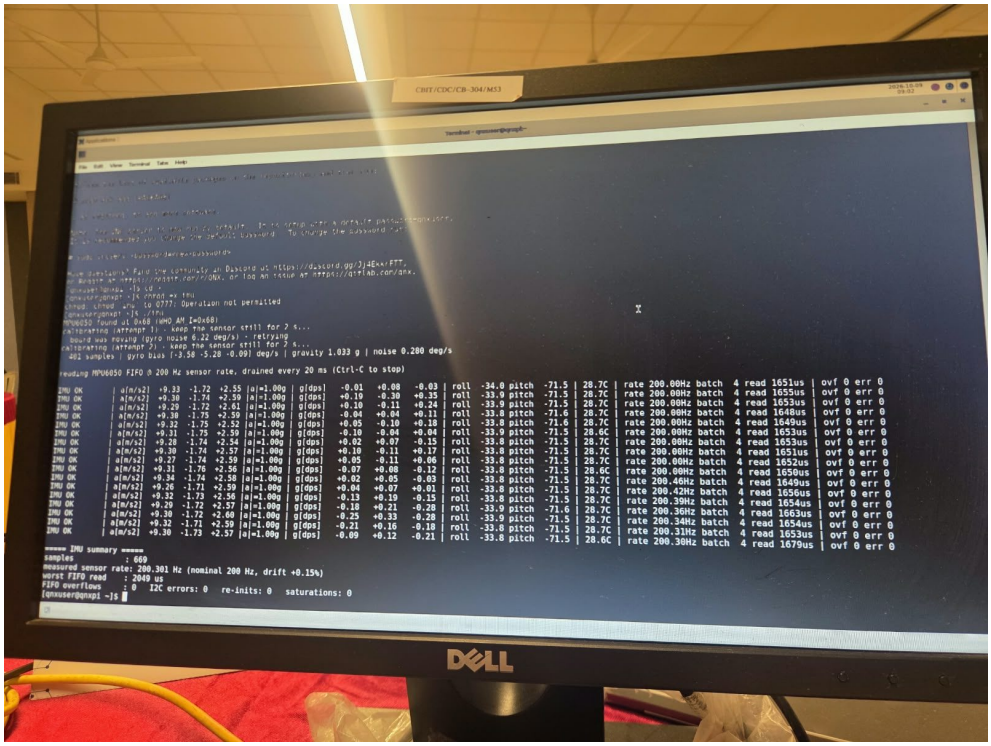



**Figure 12.** IMU node on the Pi display: calibration rejected while moving, accepted on the second attempt.

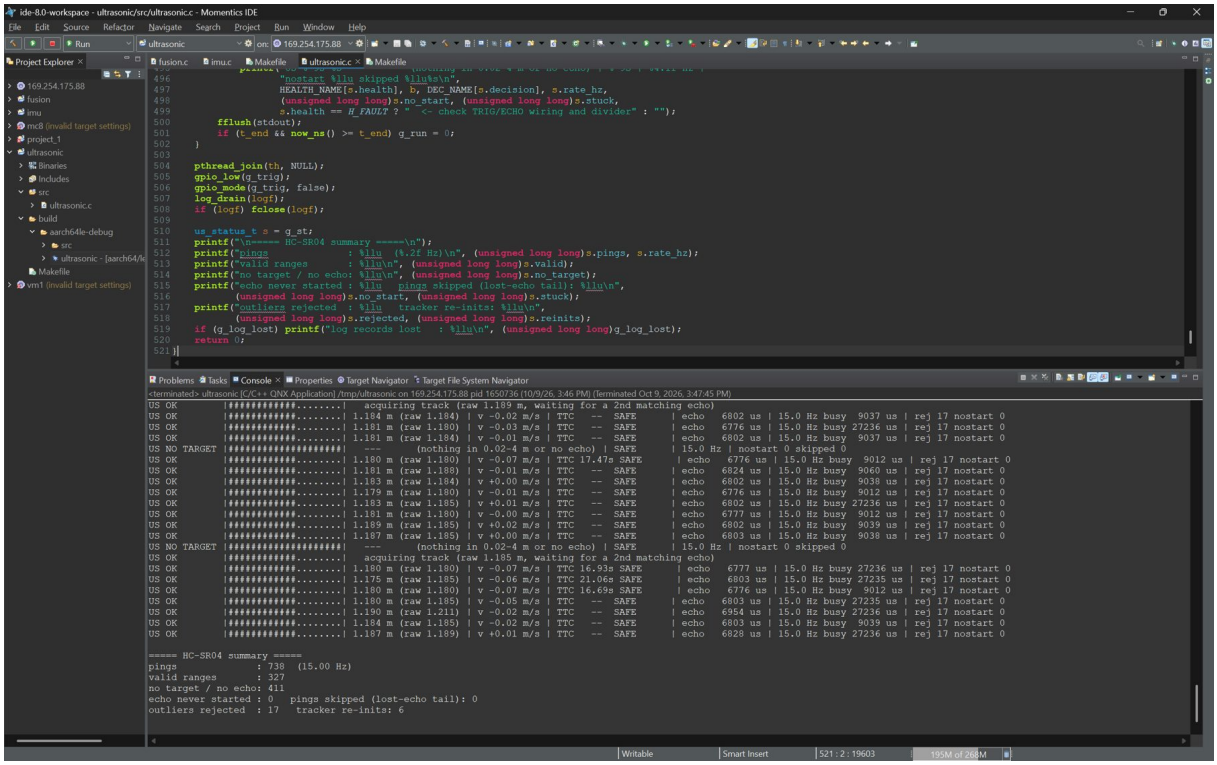



**Figure 13.** Ultrasonic node: two-echo track confirmation and lost-echo handling, 15.00 Hz.

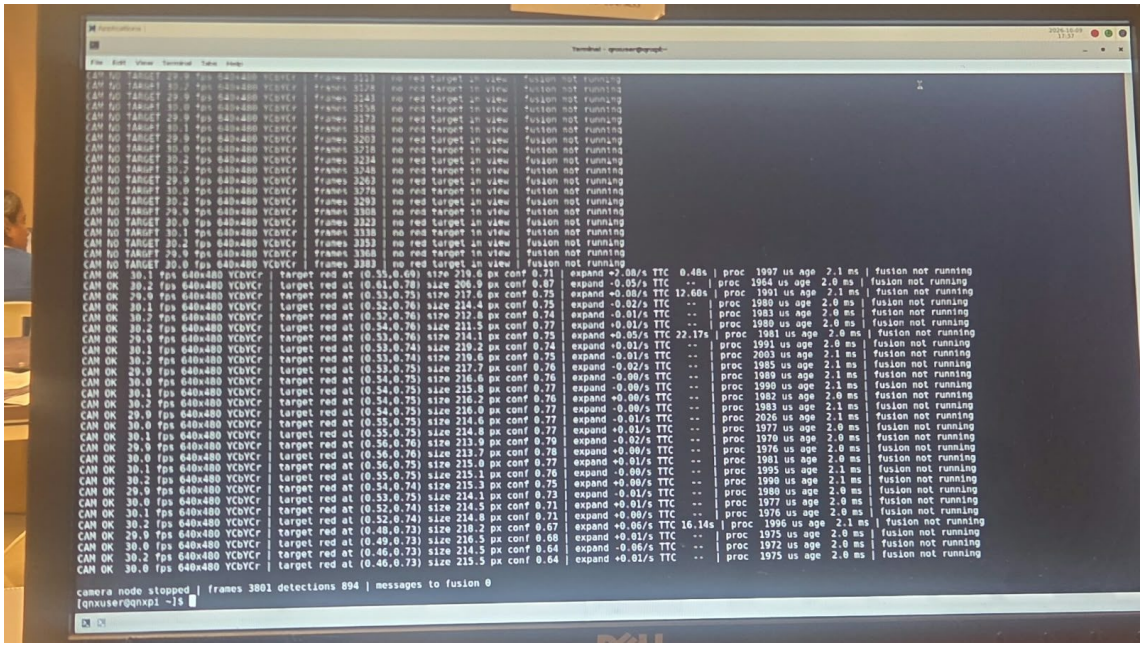


**Figure 14.** Camera node on the Pi: 30 fps, red target tracked at about 2 ms per frame.

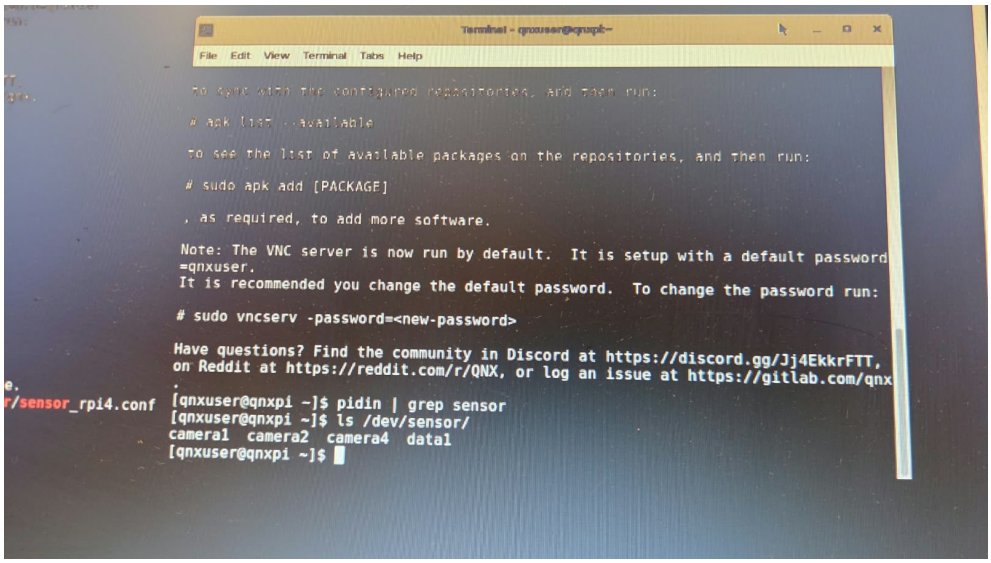



**Figure 15.** QNX Sensor Framework running: /dev/sensor/camera4 is the USB camera.

# 10. Limitations and future work

| Area | Current state | Next step |
|---|---|---|
| Object detection | colour test target | neural detection (TensorFlow Lite) as another camera process |
| Recovery | manual restart, automatic reconnect | QNX HAM to restart failed nodes automatically |
| CPU isolation | SCHED_FIFO priorities on 4 cores | runmask / adaptive partitioning for the safety loop |
| Actuation | dashboard alert | GPIO buzzer / LED, CAN-FD message to a brake ECU |
| Ultrasonic range | reliable to about 1.5 m with this module | better transducer or radar |
| Validation | bench tests | logged drive scenarios, plots from fusion_log.csv |

# 11. Development notes

The team designed the system, wired and tested the hardware, ran and debugged every component on the Raspberry Pi, and collected the measurements in this report. Parts of the source code and of this documentation were drafted with the assistance of an AI assistant (Claude) and then reviewed, built and tested by the team.

# Appendix A. Repository layout

```
qnx-adas-sensor-fusion/
  common/adas_msg.h            shared message, pulse and shared-memory definitions
  fusion/      src/fusion.c      Makefile
  imu/         src/imu.c         Makefile
  ultrasonic/  src/ultrasonic.c  Makefile
  camera/      src/camera.c      Makefile   (LIBS += -lcamapi -lm)
  dashboard/   src/dashboard.c   Makefile
  scripts/run_all.sh           start-up script for the Pi
  docs/                        architecture diagrams, presentation, profiler trace,
                               IMU guide, this report
```

# Appendix B. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| undefined reference to sqrt | math library not linked | LIBS += -lm |
| camera/camera_api.h not found | camera library not installed | install the Sensor Framework Camera Library package |
| mmap GPIO registers failed | ultrasonic needs root | run with sudo |
| Permission denied on .out file | redirect done by non-root shell | sudo sh -c '... > file 2>&1 &' |
| name_attach error | fusion already running | sudo slay fusion |
| camera_open failed | sensor service not running | start sensor (section 7.3) |
| $'\r' or not found in script | Windows line endings | run the commands by hand or convert to LF |
| Ultrasonic mostly NO TARGET | soft/angled target, interference | flat hard target, 0.2-1.5 m, sensor raised |